

Introduction to DATASCI 406

Mark M. Fredrickson (mfredric@umich.edu)

Computational Methods in Statistics and Data Science (DATASCI 406)

Course Overview

Course Goals

At the end of this course you will be able to

- Trade **computation time** for **analytical effort**
- Connect **mathematical theory** to **computational applications**
- Understand and explain **costs, benefits, and limits** of computational techniques
- Implement key **statistical algorithms** in R
- Create your own **statistical models**
- **Analyze data** using computational techniques

What is Computational Statistics?

Statistics: the application of **probability theory** to real **data**.

Goals:

- Specifying **models of data generation**
- Performing **inference** to connect real data to model
 - **Estimate** model parameters
 - **Test** hypotheses
 - **Predict** future observations
- Quantify **uncertainty** in the inference process.
- Understand **operating characteristics** of tools.

Computational statistics uses **substantial amounts of computation** to achieve these goals.

Two fundamental concepts in computational statistics

Monte Carlo: replace a quantity you cannot compute with an average over random draws. If we can get a sample $X_1, \dots, X_n$, then

$$E(g(X)) = \int g(x)f(x) dx \approx \frac{1}{n} \sum_{i=1}^n g(X_i)$$

Optimization: search for the parameter value that makes something best, when you cannot solve for it algebraically.

E.g., for a complicated density f with parameter θ , what value of θ would be most consistent with my sample?

Four Modules

Module	Name	Topic
1	Integrate	Monte Carlo with the distributions you have
2	Generate	Monte Carlo when you need a new distribution
3	Resample	Monte Carlo with only your data
4	Fit	Optimization

Let's see some examples!

Module 1: Integrate

A density

Consider a **random variable** X with density f :

$$f(x) = \frac{1}{\sqrt{2\pi}} \exp \left\{ 1.5x - 0.25x^2 - 2.5966 \right\}$$

How would you find $E(X) = \int_{-\infty}^{\infty} xf(x) dx$?

Nothing here looks like an integral you can do by hand.

Draw, then average

Suppose I hand you `sampleX(n)`, which returns n draws from f .

```
x <- sampleX(100000)
mean(x)
```

```
## [1] 2.999
```

More complicated integrals

$$E(|X|) = \int_{-\infty}^{\infty} |x| f(x) dx$$

$$\Pr(X > 5) = \int_5^{\infty} f(x) dx$$

$$E(\log(1 + e^X)) = \int_{-\infty}^{\infty} \log(1 + e^x) f(x) dx$$

$$E((1 + X^2)^{-1}) = \int_{-\infty}^{\infty} \frac{f(x)}{1 + x^2} dx$$

Same code, different g

Each is $E(g(X)) = \int g(x)f(x) dx$ for a different g .

```
c(EabsX = mean(abs(x)),  
  P_gt_5 = mean(x > 5),  
  Esoft = mean(log1p(exp(x))),  
  Einv = mean(1 / (1 + x^2)))
```

```
##   EabsX  P_gt_5   Esoft   Einv  
## 3.01620 0.07904 3.10634 0.17436
```

Change g , keep everything else.

Sampling in R

Every common distribution has an `r` function:

<code>rnorm</code>	Normal	<code>rbinom</code>	Binomial
<code>runif</code>	Uniform	<code>rpois</code>	Poisson
<code>rexp</code>	Exponential	<code>rgamma</code>	Gamma
<code>rt</code>	Student's t	<code>rbeta</code>	Beta

All take the number of draws first: `rnorm(n, mean, sd)`.

```
sampleX
```

```
## function (n)
```

```
## rnorm(n, mean = 3, sd = sqrt(2))
```

Functions of samples

A **statistic** is a function of a sample:

$$T = T(X_1, X_2, \dots, X_n)$$

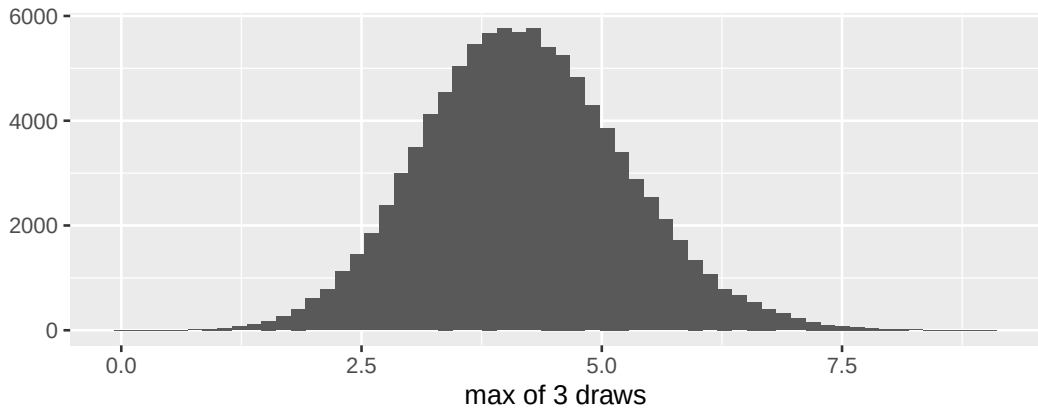
We use statistics to

- Estimate population parameters
- Test hypotheses about populations
- Perform prediction for new observations

A statistic is a random variable, so it has a distribution.

$$T = \max_{i=1,2,3} X_i$$

```
sample_max <- function(n) max(sampleX(n))  
empirical_distrib <- replicate(100000, sample_max(3))
```



Some Properties

```
mean(empirical_distrib); var(empirical_distrib)
```

```
## [1] 4.195
```

```
## [1] 1.124
```

```
quantile(empirical_distrib, c(0.25, 0.5, 0.75))
```

```
##    25%    50%    75%
```

```
## 3.465 4.157 4.882
```

These are **estimates**. How far off? That is module 1.

Module 2: Generate

A distribution R does not have

The **Pareto** distribution: incomes, city sizes, file sizes. A heavy right tail, so extreme values are far more common than under a Normal.

For shape $\alpha > 0$ and $x \geq 1$:

$$\Pr(X > x) = x^{-\alpha}$$

Therefore it has a **cumulative distributiun function** (CDF)

$$F(x) = 1 - x^{-\alpha}, \quad x \geq 1$$

The quantile function

Given a probability u , which x has $F(x) = u$?

$$u = 1 - x^{-\alpha} \implies x^{-\alpha} = 1 - u \implies F^{-1}(u) = (1 - u)^{-1/\alpha}$$

$F^{-1}(0.5)$ is the median, $F^{-1}(0.9)$ the 90th percentile.

```
qpareto <- function(u, alpha = 3) (1 - u)^(-1/alpha)
qpareto(c(0.5, 0.9, 0.99))
```

```
## [1] 1.260 2.154 4.642
```

How to sample? Inversion!

Later we will prove: If $U \sim \text{Unif}(0, 1)$, then $F^{-1}(U)$ has CDF F .

Algorithm in R:

- Sample U using `runif`
- Create X using $F^{-1}(U)$

A Pareto sampler in three lines

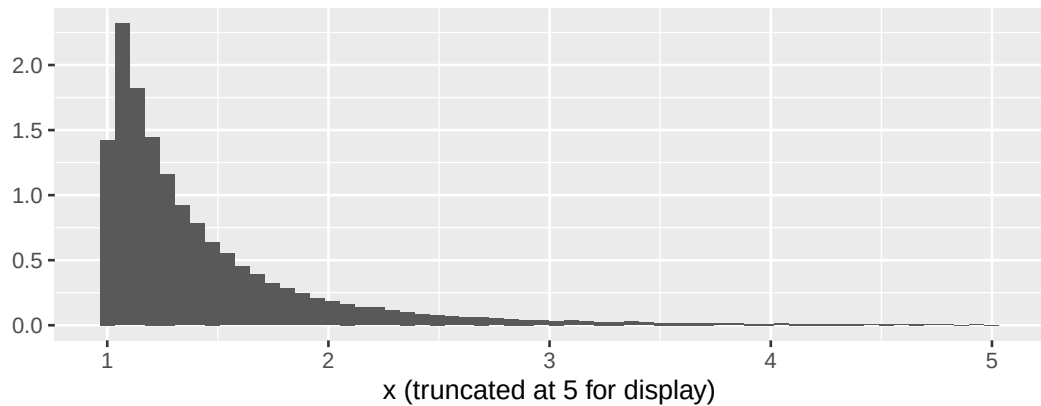
```
rporeto <- function(n, alpha = 3) qpareto(runif(n), alpha)
```

```
x <- rporeto(100000)
```

```
c(mean = mean(x), exact = 3 / 2, min = min(x))
```

```
## mean exact min
```

```
## 1.502 1.500 1.000
```



Module 3: Resample

Unknown distributions

In the previous examples, we had **known distributions** (later we will see these in hypothesis tests and estimating parameters of assumed population distributions).

What about situations where we **don't want to assume a population distribution?**

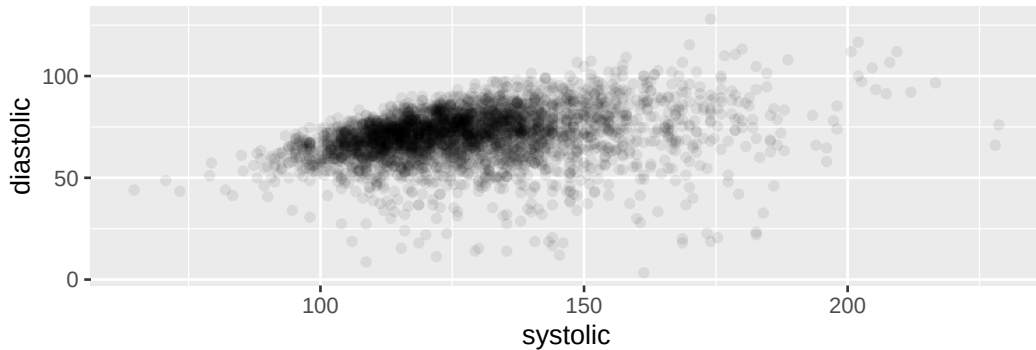
Idea: Instead of writing down a CDF F , what if we estimated it?

What distribution is this?

Systolic and diastolic blood pressure, NHANES 2013–14.

```
load("../..data/aspirin.rda")  
dim(combined)
```

```
## [1] 3554    5
```



No named family fits

```
## systolic diastolic  
##    0.8871    -0.6222
```

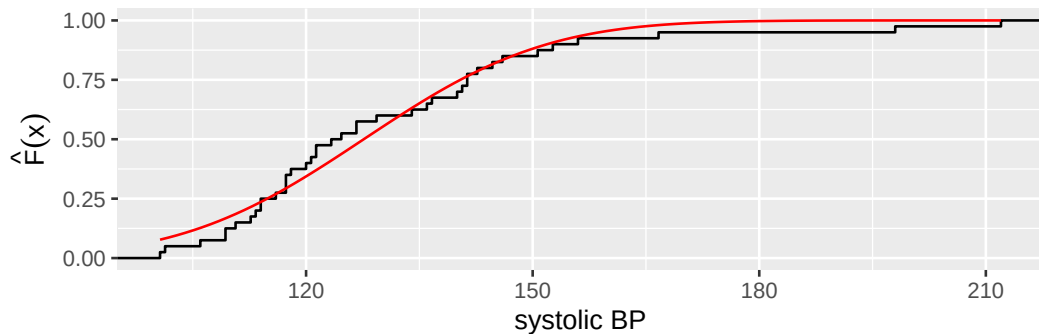
The two margins skew in **opposite directions**, so this is not bivariate Normal. There is also a clump of implausibly low diastolic readings.

We could go hunting for a family that fits. Or we could stop assuming one.

Estimating the CDF

The **empirical CDF** puts mass $1/n$ on each observed value:

$$\hat{F}(x) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{X_i \leq x\}$$



Inversion, one module later: The Bootstrap

Module 2: to sample from F , apply F^{-1} to a uniform draw.

Now do exactly that to $\hat{F}$. Since $\hat{F}$ puts mass $1/n$ on each observed point, $\hat{F}^{-1}(U)$ returns **one of your data points, chosen uniformly at random**.

sampling from $\hat{F} = \text{sample}(x, \text{replace} = \text{TRUE})$

We call this “bootstrapping.” It is inversion applied to the CDF you estimated.

Correlation of systolic and diastolic BP

Blood pressure, NHANES 2013–14:

```
theta <- cor(combined$sys_mean, combined$dia_mean)
c(n = nrow(combined), r = theta)
```

```
##           n           r
## 3554.000    0.293
```

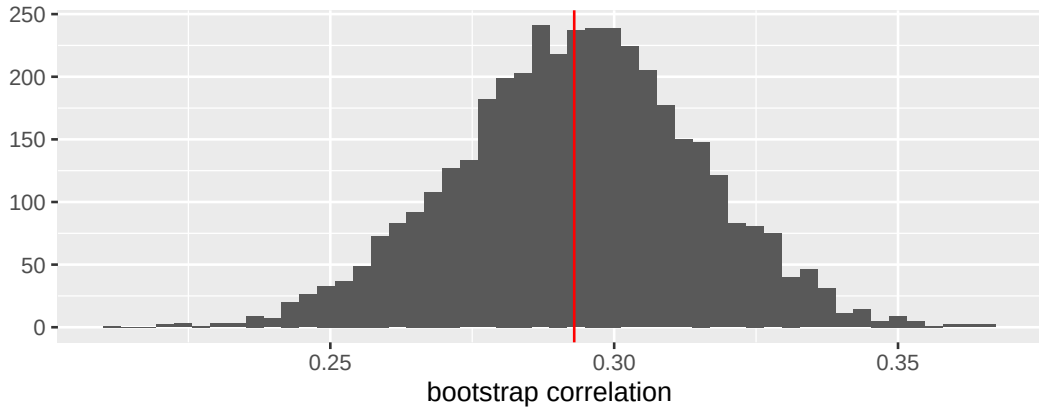
What is the **standard error of that correlation**? What would be a good **95% confidence interval**?

Bootstrapping for SE, quantiles

Treat the sample as the population and draw from it.

```
set.seed(406)
boot_r <- replicate(4000, {
  i <- sample(nrow(combined), replace = TRUE)  # rows, not values
  cor(combined$sys_mean[i], combined$dia_mean[i])
})
c(se = sd(boot_r), quantile(boot_r, c(0.025, 0.975)))
```

```
##      se    2.5%   97.5%
## 0.02137 0.25020 0.33478
```



Module 4: Fit

Only the maximum survives

Same $X \sim f$ as before, $\sigma^2 = 2$ known. We draw $m = 200$ groups of three, but the record keeps only the largest value in each group:

$$M_j = \max(X_{j1}, X_{j2}, X_{j3}), \quad j = 1, \dots, 200$$

```
set.seed(406)
m <- 200
groups <- matrix(sampleX(m * 3), nrow = m)
M <- apply(groups, 1, max)  # all we get to see
```

Estimate μ .

How to estimate?

Note that μ is not the same as $E(\max X)$:

```
mean(M)
```

```
## [1] 4.299
```

1. Maximum likelihood estimation
2. Algorithm that converges to the MLE

Maximum likelihood, briefly

The **likelihood** is the probability of the data we saw, read as a function of the unknown parameter:

$$L(\mu) = \prod_{j=1}^m f_M(M_j; \mu) \qquad \hat{\mu} = \arg \max_{\mu} L(\mu)$$

Pick the μ that makes what we observed as likely as possible. More on this in the statistical review.

Why not just maximize the likelihood?

The max of three draws has density $3f(x)F(x)^2$, so

$$\ell(\mu) = \sum_{j=1}^m \left[\log \phi\left(\frac{M_j - \mu}{\sigma}\right) + 2 \log \Phi\left(\frac{M_j - \mu}{\sigma}\right) \right]$$

Setting $\ell'(\mu) = 0$ gives

$$\sum_{j=1}^m \frac{M_j - \mu}{\sigma^2} = \frac{2}{\sigma} \sum_{j=1}^m \frac{\phi\left(\frac{M_j - \mu}{\sigma}\right)}{\Phi\left(\frac{M_j - \mu}{\sigma}\right)}$$

μ is stuck inside Φ , which is itself an integral with no closed form. **You cannot solve this for μ .**

What we would do with the full data

With all $3m$ values in hand, estimating μ is a sample mean:

$$\hat{\mu} = \frac{1}{3m} \sum_{j=1}^m \underbrace{\sum_{i=1}^3 X_{ji}}_{S_j}$$

We only need the **group totals** S_j , not the individual values.

We cannot compute S_j . But we know one of its three terms and have a bound on the other two.

Making a reasonable guess

Replace the total we cannot compute with **its expectation given what we saw**.

Given M_j , one value equals M_j and the other two are draws from f conditioned to lie below M_j :

$$E(S_j | M_j) = M_j + 2E(X | X \leq M_j)$$

For a Normal the conditional mean is closed form:

$$E(X | X \leq a) = \mu - \sigma \frac{\phi\left(\frac{a-\mu}{\sigma}\right)}{\Phi\left(\frac{a-\mu}{\sigma}\right)} < \mu$$

It depends on μ , which is what we are trying to find. So guess, then iterate.

Example

Start from the observed average maximum, $\mu^{(0)} = 4.30$, and take the first group, which came back with $M_j = 5.77$:

$$E(X \mid X \leq 5.77) = 3.91$$

$$E(S_j \mid M_j) = 5.77 + 2(3.91) = 13.60$$

The true total was $1.16 + 2.57 + 5.77 = 9.50$.

One group is badly wrong. The M step averages 200 of them.

EM: Expectation-Maximization algorithm

Start with a guess $\mu^{(0)}$. For $t = 1, 2, \dots$:

1. **E**: compute $E(S_j \mid M_j)$ for every group, using $\mu^{(t-1)}$.
2. **M**: set $\mu^{(t)} = \frac{1}{3m} \sum_j E(S_j \mid M_j)$.
3. Repeat until it **convergence** (estimate stops moving).

Starting from $\mu^{(0)} = 4.30$, one step gives 3.49.

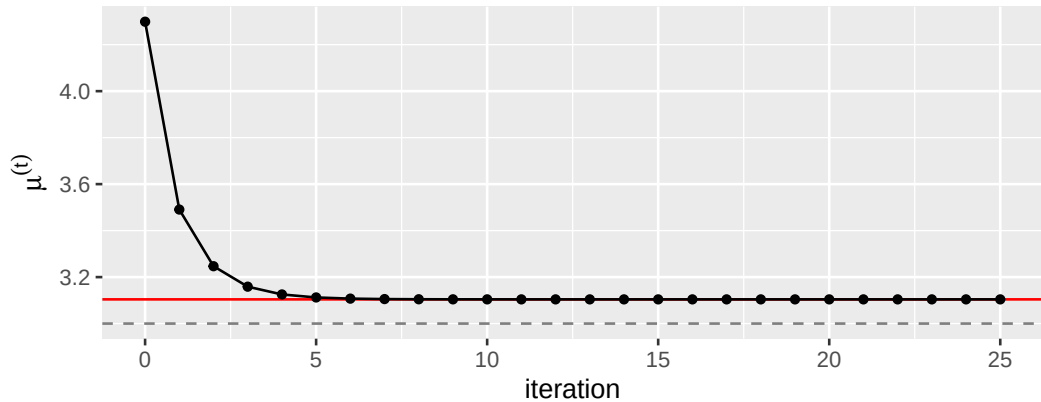
EM in R

```
sigma <- sqrt(2)
Etrunc <- function(a, mu, s) mu - s * dnorm((a-mu)/s) / pnorm((a-mu)/s)

guess <- mean(M)
for (i in 1:100) {
  e_step <- (M + 2 * Etrunc(M, guess, sigma)) / 3
  m_step <- mean(e_step)
  guess <- m_step
}
c(EM = m_step, naive = mean(M), mu = 3)
```

```
##      EM naive      mu
## 3.104 4.299 3.000
```

Convergence



Red is the **MLE from the data we actually have**; dashed grey is $\mu = 3$.

What is EM converging to?

Not μ , and not the average of the values we lost. EM maximizes the likelihood of the data **we observed**:

population μ	3.000
mean of the 600 hidden values	3.063
MLE from the 200 maxima	3.104
EM after 25 steps	3.104

EM lands on the MLE, exactly. The gap from 3 is **sampling error in 200 maxima**, not a flaw in the algorithm. With more groups the MLE moves toward 3.

Course Logistics

Professor Mark Fredrickson (<mfredric@umich.edu>)

GSIs Ishan Paul (<ishanpl@umich.edu>), Qingyuan Yu
(<qyyu@umich.edu>)

Student Hours TBA, see Canvas

Prerequisites

- Some programming experience (R, Python, C/C++, Java)
- An understanding of the following terms: Distributions and random variables, random sampling and sampling distributions, population parameters and estimation, hypothesis tests, basic calculus (integrals, derivatives).

No required text, but several **recommended** (particularly first 2). Links to electronic copies available through the library on Canvas.

- Rizzo, M. L. *Statistical Computing with R*, 2nd ed.
- Wickham, H. & Grolemund, G. *R for Data Science*
- Robert, C. P. & Casella, G. *Introducing Monte Carlo Methods with R*
- Irizarry, R. A. *Introduction to Data Science*
- Agresti, A. *Foundations of Linear and Generalized Linear Models*
- Gentle, J. E. *Computational Statistics*
- Härdle, W., Gentle, J. E. & Mori, Y., eds. *Handbook of Computational Statistics*

All slides and videos will be posted to **Canvas** and **OneNote**.

Approach

The class is composed of **four modules** and uses a **flipped classroom** approach.

Each module consists of short **check-in quizzes**, a **project**, and an **exam**. There is no separate final exam.

Materials are presented in **video lectures**. Class time is for short **check-in quizzes**, working on examples and the project, and exams.

Quizzes

- 4–6 multiple choice plus one short answer, every session
- Covers the material you watched **before** class
- **Lowest per module dropped. No makeups.**
- I will reach out to students with testing accommodations.

Each module includes a project. The project gives you an opportunity to:

- Implement key statistical algorithms
- Investigate the properties of those algorithms
- Apply these techniques to real-world data

These projects become your notes for your exams. **You will be asked questions on the exams about your project.**

Exams

Each module concludes with an exam. Approximate coverage:

- 50%: General conceptual questions, similar to the quizzes.
- 50%: Questions about the results of your own project.

You may bring any **paper** notes you wish. No electronic devices. Held during class periods, with the exception of the Module 4 exam, which uses the scheduled final exam time.

Students with accommodations will be contacted with alternate exam location information.

Each module counts for 25% of your grade. Within each module:

- Exam: 20% of overall grade
- Quizzes: 5% of overall grade, one drop per module

Permitted and expected on projects and all out-of-class work.

Not permitted on quizzes or exams.

Remember: you will be asked about project details. Be prepared to explain coding details, even if you used AI to assist.

Other Policies

- Course Recordings
- Disability Statement
- Mental Health and Well-Being
- Sexual Misconduct Policy

Next Session

- No quiz
- Video: statistical review
- Class period: Introduction to R and RMarkdown
- All students: install R and RStudio on laptops, directions on Canvas